NLP and Word Embeddings

Structured Notes

Text Pre-processing Pipeline,

Vocabulary Creation,
One-Hot Encoding, and Introduction to Word Embeddings

Contents

1 Introduction to Natural Language Processing **2**
1.1 Why NLP is Different from Other Data Types . . . 2
1.2 Challenges of Text Data . . . 2
1.3 Learning Roadmap . . . 3

2 Text Pre-processing Pipeline **3**
2.1 Why Pre-processing is Required . . . 3
2.2 Pipeline Overview . . . 3
2.3 Tokenization . . . 3
2.4 Lowercasing . . . 4
2.5 Stop Word Removal . . . 4
2.6 Stemming . . . 4
2.7 Lemmatization . . . 5
2.8 POS Tagging for Lemmatization . . . 5
2.9 Complete Pipeline Example . . . 5
2.10 Pre-processing Code Implementation . . . 6
2.11 Why Pre-processing Matters . . . 7

3 Vocabulary Creation and Word Indexing **7**
3.1 Vocabulary Creation . . . 7
3.2 Word Indexing (Dictionary Mapping) . . . 7
3.3 Problems with Numeric Indexing . . . 8

4 One-Hot Encoding **8**
4.1 How One-Hot Encoding Works . . . 8
4.2 Properties of One-Hot Encoding . . . 8
4.3 Advantages of One-Hot Encoding . . . 9
4.4 Limitations of One-Hot Encoding . . . 9

5 Introduction to Word Embeddings **9**
5.1 The Context Problem . . . 9
5.2 Evolution of Text Representation . . . 10

6 Summary **10**

1 Introduction to Natural Language Processing

Natural Language Processing (NLP)

Natural Language Processing (NLP) is the field of artificial intelligence concerned with enabling machines to understand, interpret, and process human language — specifically **text data**. Unlike numerical, image, or time series data, text requires specialized pre-processing before it can be fed into neural networks.

NLP is one of the most critical topics in the current technology landscape. Applications range from chatbots and prompt engineering to large language models (LLMs), transformers, generative AI, and retrieval-augmented generation (RAG). All of these rely fundamentally on how text data is optimized and converted into a format that machines can process.

Industry Relevance

NLP is ubiquitous in everyday applications:

- **Keyboard predictions**: WhatsApp and Google Keyboard predict the next word based on chat history
- **Chatbots**: Before GPT-based models, rule-based chatbots were built using RNN, LSTM, and GRU architectures
- **Prompt engineering**: Modern LLMs process text prompts, and understanding tokenization is essential
- **RAG systems**: RAG optimizes prompts by converting them into vector embeddings before passing to a transformer model

1.1 Why NLP is Different from Other Data Types

Previously covered data types include:

- **Images**: Pixels, which are inherently numerical
- **Tabular data**: Numbers that can be directly processed
- **Time series**: Continuous numerical signals

Text data (e.g., "I love machine learning", "Deep learning is powerful", "AI is transforming industries") consists of **non-numerical values**. Neural networks require numerical input — specifically tensors and vectors. Therefore, text must be converted into numerical representations before any model can process it.

> **Key Insight:** The conversion from text to vectors (embeddings) is the critical step that differentiates NLP from other data processing. Once text is converted into vectors, the downstream processing follows the same principles as numerical data.

1.2 Challenges of Text Data

1. **Sequential in nature**: Language is sequence-dependent. Word order matters for preserving context and meaning.
2. **Vocabulary challenge**: Given a dataset like "I love NLP, I love deep learning", the vocabulary consists of all unique words: {I, love, NLP, deep, learning}. Vocabulary size grows quickly as the corpus expands.
3. **High-dimensional problem**: If the vocabulary size is 10,000, each word must be represented in a 10,000-dimensional space. This creates:
   - Memory issues
   - Computation issues
   - Learning difficulty

This is the **curse of dimensionality**.

1.3 Learning Roadmap

The NLP learning path covers:

1. Text pre-processing
2. Vocabulary creation
3. Encoding methods (dictionary mapping, one-hot encoding)
4. Word embeddings (Word2Vec, GloVe)

This foundation prepares for RNN, LSTM, GRU, and LLM implementations. LSTM and GRU are the primary architectures used in practice, as basic RNN has known limitations.

2 Text Pre-processing Pipeline

Text Pre-processing

Text pre-processing is the pipeline of operations applied to raw text data to clean and standardize it before conversion to numerical representations. It is analogous to the data pre-processing pipeline for numerical data (handling missing values, normalization, noise removal).

2.1 Why Pre-processing is Required

Consider these three raw sentences:

- "I love Machine Learning"
- "I LOVE machine learning"
- "I love ML"

For humans, all three sentences have the same meaning. For machines, these are completely different due to case sensitivity, abbreviations, and character differences. Without pre-processing, this creates:

- **Vocabulary explosion**: Each variant is treated as a different word
- **Poor learning**: The model cannot generalize across variants
- **Data inconsistency**: Same concepts have multiple representations

2.2 Pipeline Overview

The NLP pre-processing pipeline consists of the following sequential steps:

Step	Operation	Purpose
1	Raw Text Input	Starting point
2	Tokenization	Split sentences into individual words
3	Lowercasing	Normalize case for consistency
4	Stop Word Removal	Remove non-meaningful connecting words
5	Stemming / Lemmatization	Reduce words to root forms

2.3 Tokenization

Tokenization

Tokenization is the process of converting a sentence into individual words (tokens). Mathematically,

a sentence S is decomposed as:

$$S = (w_1, w_2, w_3, \ldots, w_n)$$

where each w_i is a token (word).

Example:

- Input: "I love NLP"
- Tokens: [`I`, `love`, `NLP`]

Token limits in language models (e.g., "20,000 token limit") refer to the number of words that can be included in a prompt.

2.4 Lowercasing

All text is converted to lowercase to ensure that words like "I" and "i", or "NLP" and "nlp", are treated as the same token. This reduces vocabulary size and ensures consistency.

Example:

- Input: "I Love NLP"
- Output: "i love nlp"

2.5 Stop Word Removal

Stop Words

Stop words are common words (e.g., "is", "the", "and", "are", "of") that serve as connecting words in sentences but do not carry mathematical or semantic value. They make sentences grammatically correct but do not contribute to the core meaning.

Example:

- Input: "This is a machine learning model"
- After stop word removal: "machine learning model"

The words "this", "is", and "a" have no standalone meaning — they are connecting words that form grammatical structure. The focus is on words that carry actual semantic content.

2.6 Stemming

Stemming

Stemming reduces words to their root form by removing suffixes. It is a fast but sometimes imprecise operation.

Examples:

Original Word	Stemmed	Correct?
running	run	Yes
played	play	Yes
learning	learn	Yes
studies	studi	No (incorrect root)
better	bet	No (incorrect root)
preferably	prefer	Yes

Stemming is fast but can produce incorrect root forms because it operates purely on string manipulation without understanding word meaning.

2.7 Lemmatization

Lemmatization

Lemmatization reduces words to their dictionary root form (lemma) using word meaning and part-of-speech (POS) tagging. It is more accurate than stemming but slower.

Examples:

Original Word	Lemmatized	Correct?
running	run	Yes
better	well/good	Yes
studies	study	Yes
preferably	preferably	Yes

Stemming vs. Lemmatization

- **Stemming**: Fast, operates on string patterns, sometimes produces incorrect roots
- **Lemmatization**: Slower, uses POS tagging and word meaning, always produces valid dictionary words
- **Recommendation**: Lemmatization is preferred for accuracy. It can be used directly without applying stemming first.

2.8 POS Tagging for Lemmatization

Lemmatization requires **Part-of-Speech (POS) tagging** to correctly identify word forms. The POS tagger classifies words into categories such as:

- **Noun**: person, place, thing
- **Verb**: action word (e.g., "run", "learn")
- **Adjective**: descriptive word
- **Adverb**: modifies a verb

For example, "running" needs to be identified as a verb form so that the lemmatizer can correctly reduce it to "run". Without POS tagging, the WordNet lemmatizer cannot function correctly.

2.9 Complete Pipeline Example

Input sentence: "I am learning machine learning and loving it"

Stage	Output
Tokenization	[learning, machine, learning, loving]
Stemming	[learn, machin, learn, love]
Lemmatization	[learn, machine, learn, love]

Note that "learn" appears twice because "learning" occurs in two positions in the original sentence. This preserves the sequential context of the sentence — it is not the vocabulary but the tokenized representation.

Second example: "I enjoy learning machine learning and loving it better"

Stage	Output
Tokens	[learning, machine, learning, enjoy]
Stemmed	[learn, machin, learn, enjoy]
Lemmatized	[learn, machine, learn, enjoy]

2.10 Pre-processing Code Implementation

NLP Pre-processing Pipeline

```python
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag

# Download required NLTK resources
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')

# Initialize tools
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

# Input sentence
sentence = "I am learning machine learning and loving it"

# Step 1: Lowercase
sentence_lower = sentence.lower()

# Step 2: Tokenize (split into words)
tokens = re.findall(r'\b\w+\b', sentence_lower)

# Step 3: Remove stop words and punctuation
tokens = [w for w in tokens if w not in stop_words]

# Step 4: Stemming
stemmed = [stemmer.stem(w) for w in tokens]

# Step 5: Lemmatization with POS tagging
def get_wordnet_pos(tag):
    if tag.startswith('J'):
        return 'a'  # adjective
    elif tag.startswith('V'):
        return 'v'  # verb
    elif tag.startswith('R'):
        return 'r'  # adverb
    else:
        return 'n'  # noun (default)

pos_tags = pos_tag(tokens)
lemmatized = [
    lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    for word, tag in pos_tags
]

print("Original:", sentence)
```

```
print("Tokens:", tokens)
print("Stemmed:", stemmed)
print("Lemmatized:", lemmatized)
```

Listing 1: Complete text pre-processing pipeline using NLTK

Output:

- Tokens: [learning, machine, learning, loving]
- Stemmed: [learn, machin, learn, love]
- Lemmatized: [learn, machine, learn, love]

2.11 Why Pre-processing Matters

After pre-processing:

- **Vocabulary size reduces**: Fewer unique tokens to represent
- **Model learns better**: Consistent representations improve generalization
- **Computation reduces**: Smaller vocabulary means lower dimensionality
- **Performance improves**: Cleaner data leads to better model accuracy

3 Vocabulary Creation and Word Indexing

After pre-processing, the next step is to create a vocabulary and convert words to numerical representations that neural networks can process.

3.1 Vocabulary Creation

Vocabulary

The **vocabulary** of a dataset is the set of all unique words that appear after pre-processing. Given a corpus of sentences, the vocabulary V is:

$$V = \{w_1, w_2, w_3, \ldots, w_n\} \quad \text{where each } w_i \text{ is unique}$$

Example:

- Dataset: “I love NLP”, “I love deep learning”
- Vocabulary: {I, love, NLP, deep, learning}
- Vocabulary size: $|V| = 5$

3.2 Word Indexing (Dictionary Mapping)

Each unique word in the vocabulary is assigned a numerical index:

Word Indexing Function:

$$f(w_i) = i \quad \text{where } i \text{ is the index of word } w_i \text{ in the vocabulary}$$

Example mapping:

Word	Index
I	0
love	1
NLP	2
deep	3
learning	4

A sentence $S = (w_1, w_2, w_3)$ becomes its numeric representation $(f(w_1), f(w_2), f(w_3))$.

Example: "I love NLP" $\rightarrow (0, 1, 2)$

3.3 Problems with Numeric Indexing

Simple numeric indexing introduces **false relationships**:

- If "king" $= 0$, "queen" $= 1$, "man" $= 2$, "woman" $= 3$
- The model might interpret "queen $>$ king" (since $1 > 0$), which is meaningless
- Numeric values create false ordinal relationships and hierarchies
- The model learns wrong patterns based on arbitrary index assignments

Key Problem

Dictionary mapping converts text to numbers, but these numbers are **meaningless** — they carry no contextual or semantic information. The numeric relationships are entirely arbitrary and can mislead the model.

4 One-Hot Encoding

One-Hot Encoding

One-Hot Encoding represents each word as a binary vector where exactly one position is 1 and all others are 0. The vector dimension equals the vocabulary size $|V|$. This eliminates the false ordering problem introduced by simple numeric indexing.

4.1 How One-Hot Encoding Works

Given vocabulary: {king, queen, man, woman} with $|V| = 4$:

One-Hot Representations:

$$\text{king} = \begin{bmatrix}1\\0\\0\\0\end{bmatrix}, \quad \text{queen} = \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, \quad \text{man} = \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, \quad \text{woman} = \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$$

4.2 Properties of One-Hot Encoding

- Only **one position** is 1; all others are 0
- Vector size equals vocabulary size: $\dim(\vec{v}) = |V|$
- Produces **sparse vectors** (mostly zeros)
- Each word has a unique vector representation

Mathematical Representation:

For vocabulary size V, each word w_i is represented as a one-hot vector $\vec{e}_i \in \mathbb{R}^V$ where:

$$(\vec{e}_i)_j = \begin{cases} 1 & \text{if } j = i \\ 0 & \text{otherwise} \end{cases}$$

4.3 Advantages of One-Hot Encoding

- **No numeric bias**: No false ordering between words
- **Unique representation**: Each word gets a distinct vector
- **No false meaning**: Since all vectors have the same dimension and the same number of 1s, no word appears "larger" or "smaller" than another

4.4 Limitations of One-Hot Encoding

Critical Limitation

One-hot encoding **cannot capture semantic similarity**. For example:

- The cosine similarity between "king" and "queen" is 0
- But "king" and "queen" are semantically related concepts
- Every pair of words has identical (zero) similarity in one-hot space

This is because one-hot vectors are **orthogonal** to each other — no two vectors share any common dimension.

Additionally, as vocabulary size grows, the one-hot vectors become extremely high-dimensional, leading back to the **curse of dimensionality**. For a vocabulary of 10,000 words, each word requires a 10,000-dimensional vector, most of which is zeros.

5 Introduction to Word Embeddings

Word Embeddings

Word embeddings are dense, low-dimensional vector representations of words that capture semantic relationships. Unlike one-hot encoding (sparse, high-dimensional, no similarity), word embeddings place semantically similar words close together in vector space.

The limitations of one-hot encoding — inability to capture similarity, curse of dimensionality, and context-independent representations — motivate the need for word embeddings.

5.1 The Context Problem

Consider the word "bank":

- "I am going to the **river bank**" — refers to the edge of a river
- "I am going to the **financial bank**" — refers to a financial institution

The same word has completely different meanings depending on context. One-hot encoding assigns the same vector to "bank" regardless of context, losing this crucial distinction. Word embeddings address this by learning representations that encode contextual meaning.

5.2 Evolution of Text Representation

Era	Method	Limitation
Early NLP	Dictionary mapping	False numeric relationships
Early NLP	One-hot encoding	No similarity, high dimensionality
Modern NLP	Word embeddings (Word2Vec, GloVe)	Static context
Current NLP	Contextual embeddings (Transformers)	Compute-intensive

Word embeddings (Word2Vec, GloVe) and contextual embeddings represent the modern approach to text representation, forming the foundation for transformers and large language models.

6 Summary

- **NLP** processes text data, which is fundamentally different from numerical, image, or time series data because neural networks require numerical input
- The **text pre-processing pipeline** consists of: tokenization → lowercasing → stop word removal → stemming/lemmatization
- **Tokenization** splits sentences into individual word tokens
- **Stop word removal** eliminates non-meaningful connecting words to reduce vocabulary
- **Stemming** is fast but sometimes produces incorrect root forms; **lemmatization** uses POS tagging for accurate root-form identification and is preferred
- **Vocabulary** is the set of unique words; **word indexing** assigns numeric indices to each word
- **Dictionary mapping** creates false numeric relationships between words
- **One-hot encoding** eliminates false ordering but cannot capture semantic similarity and suffers from the curse of dimensionality
- **Word embeddings** are the solution — dense, low-dimensional vectors that capture semantic relationships and contextual meaning
- This NLP foundation is essential for understanding RNN, LSTM, GRU, transformers, and LLM architectures